

Control de posición de motor DC

Evaluación Globalizadora A - Informe

Lucas Ignacio Gonzalez

lucassignaciogonzalez1999@gmail.com

Fernando Jorja

quilmesmakeup@hotmail.com

Universidad Tecnológica Nacional - Facultad Regional Avellaneda
Técnicas Digitales III

Abstract— El presente informe describe el desarrollo de un sistema embebido para el control de posición angular de un motor de corriente continua. El proyecto utiliza un microcontrolador ESP32-S3 como unidad de procesamiento, un encoder magnético AS5600 para la medición de posición, un driver L298N para el accionamiento del motor y un firmware basado en ESP-IDF con FreeRTOS. El sistema propuesto permitirá ingresar un ángulo objetivo, medir la posición real del eje, calcular el error angular y accionar el motor mediante PWM y señales de dirección. Además, se prevé la implementación de un controlador PID, perfiles de movimiento, configuración local mediante pulsadores y display LCD I2C, configuración remota mediante UART, almacenamiento en memoria no volátil y detección de bloqueos durante el movimiento.

Palabras Clave— FreeRTOS, ESP32-S3, PID, motor DC, AS5600, PWM, I2C, UART, L298N.

I. INTRODUCCIÓN

El proyecto consiste en realizar un control de posición de un motor de corriente continua. A diferencia de un motor DC utilizado en lazo abierto, donde solo se controla el sentido de giro y la velocidad aproximada, en este caso se busca que el sistema pueda alcanzar una posición angular determinada por el usuario.

Para ello se plantea un lazo cerrado de control: el usuario define un ángulo objetivo, el sistema mide el ángulo real del eje mediante un encoder magnético y el firmware calcula la acción de control necesaria para mover el motor hasta la posición deseada. El control del motor se realizará mediante una señal PWM aplicada al driver L298N, mientras que el sentido de giro se definirá por salidas digitales del ESP32-S3.

El firmware se desarrollará utilizando ESP-IDF y FreeRTOS. La elección de un sistema operativo en tiempo real permite organizar el programa en tareas, asignar prioridades, bloquear tareas cuando no tienen trabajo pendiente y comunicar módulos mediante colas, semáforos y eventos.

II. FUNDAMENTACIÓN

A. Problemática

Un motor DC convencional no posee, por sí mismo, una referencia absoluta de posición. Si se lo alimenta directamente, el eje gira en un sentido determinado y a una velocidad dependiente de la tensión, la carga y las características del motor. Esto resulta insuficiente cuando se requiere ubicar el eje en un ángulo específico, repetir movimientos o corregir perturbaciones mecánicas.

Además, al trabajar con posiciones angulares de 0° a 360° , el cálculo del error no puede realizarse únicamente como una resta lineal. Por ejemplo, pasar de 350° a 10° no debería interpretarse como un error de -340° , sino como un desplazamiento de $+20^\circ$ por el camino angular más corto.

B. Solución propuesta

La solución propuesta consiste en cerrar el lazo de control mediante un encoder magnético AS5600 acoplado al eje del motor. El sensor entrega una medición absoluta de posición

angular, que será leída por el ESP32-S3 mediante el bus I2C. Con esa información, el firmware calculará el error angular normalizado entre -180° y $+180^\circ$, ejecutará un algoritmo PID y generará la salida de control correspondiente.

La salida del controlador se interpretará en dos partes: el signo indicará el sentido de giro y el módulo determinará el valor de PWM aplicado al driver L298N. De esta forma se podrá controlar tanto la dirección como el esfuerzo aplicado al motor.

C. Fundamentación técnica del software

El uso de FreeRTOS permite dividir el firmware en tareas con responsabilidades específicas. Las tareas críticas, como la lectura del encoder, la administración del bus I2C y el cálculo PID, se plantean con mayor prioridad que las tareas de interfaz, visualización y almacenamiento. Esta separación facilita la depuración, mejora la mantenibilidad del código y permite que cada tarea permanezca bloqueada cuando no tiene acciones pendientes.

Para la comunicación entre tareas se utilizarán colas de FreeRTOS. Por ejemplo, las tareas de interfaz local y UART podrán enviar comandos a una cola de control recibida por la tarea PID. Las interrupciones de los pulsadores no ejecutarán procesamiento extenso, sino que liberarán semáforos para despertar a la tarea encargada del manejo de botones.

D. Fundamentación técnica del hardware

El ESP32-S3 fue seleccionado como unidad central por su capacidad de ejecutar FreeRTOS, disponer de periféricos I2C, UART, GPIO y PWM, y contar con memoria no volátil interna para guardar configuraciones. El AS5600 permite medir la posición angular absoluta del eje con resolución interna de 12 bits. El L298N permite accionar el motor como puente H, definiendo sentido de giro y velocidad mediante señales digitales y PWM.

III. OBJETIVOS

El objetivo principal del proyecto es desarrollar un sistema embebido capaz de controlar la posición angular de un motor DC mediante un lazo cerrado de control.

- Permitir el ingreso o modificación del ángulo deseado.

- Medir la posición actual del eje mediante el encoder magnético AS5600.
- Calcular el error angular por el camino más corto entre la posición actual y el setpoint.
- Implementar un controlador PID ajustable.
- Accionar el motor mediante PWM y señales de dirección sobre el driver L298N.
- Incorporar perfiles de movimiento.
- Detectar condiciones de bloqueo y realizar una acción de recuperación.
- Permitir configuración local mediante botones y display LCD I2C.
- Permitir configuración mediante UART.
- Guardar parámetros de configuración en memoria no volátil.

IV. DIAGRAMA EN BLOQUES

La Fig. 1 muestra la estructura general del sistema. El ESP32-S3 concentra el procesamiento, lee el encoder AS5600 por I2C, controla el L298N mediante PWM y señales de dirección, recibe entradas desde pulsadores y permite comunicación remota por UART. Todos los módulos deben compartir una referencia de masa común.

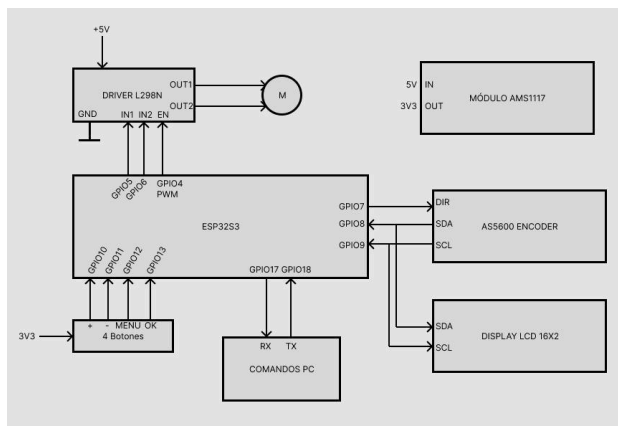


Fig. 1. Diagrama en bloques preliminar del sistema.

V. HARDWARE SELECCIONADO

A. Microcontrolador ESP32-S3

El ESP32-S3 será utilizado como unidad central del sistema. Sus funciones principales serán ejecutar FreeRTOS, leer el AS5600 por I2C, generar la señal PWM para el motor, controlar los pines de dirección del L298N, leer pulsadores mediante interrupciones GPIO, comunicarse con una PC mediante UART, manejar el display LCD I2C y almacenar configuración en NVS.

B. Motor DC

Se utilizará un motor DC de baja tensión, con un imán acoplado al eje para permitir la medición angular mediante el AS5600. Al no poseer control de posición incorporado, el sistema deberá cerrar el lazo utilizando la medición del sensor.

C. Driver L298N

El motor será accionado mediante un módulo L298N configurado como puente H. La entrada ENA recibirá la señal PWM generada por el ESP32-S3, mientras que IN1 e IN2 definirán el sentido de giro.

D. Encoder AS5600

El AS5600 es un encoder magnético absoluto que permite medir posiciones de 0° a 360° mediante comunicación I2C.

La correcta alineación del imán con el eje y con el sensor será un punto crítico para obtener lecturas estables.

E. Display, pulsadores y UART

La interfaz local estará formada por un display LCD con expensor I2C PCF8574 y cuatro pulsadores: +, -, MENU y OK. Los pulsadores se conectarán a GPIO con interrupciones, y la ISR solo liberará semáforos para diferir el procesamiento a una tarea. Además, se incorporará una interfaz UART TTL de 3,3 V para configurar parámetros desde una PC.

VI. PINES PROPUESTOS

- GPIO17 y GPIO18: UART TX/RX hacia adaptador USB-UART TTL de 3,3 V.
- GPIO10, GPIO11, GPIO12 y GPIO13: pulsadores +, -, MENU y OK.
- GPIO9: I2C SCL para AS5600 y LCD.
- GPIO8: I2C SDA para AS5600 y LCD.
- GPIO7 o GND: selección DIR del AS5600.
- GPIO6: dirección motor 2 hacia IN2 del L298N.
- GPIO5: dirección motor 1 hacia IN1 del L298N.
- GPIO4: PWM motor hacia ENA del L298N.

VII. CONTROL PID Y ERROR ANGULAR

El controlador PID calculará la salida de control a partir del error angular normalizado. La ecuación conceptual utilizada será:

$$\text{salida} = K_p * \text{error} + K_i * \text{integral} + K_d * \text{derivada}$$

El signo de la salida determinará el sentido de giro, mientras que el valor absoluto se utilizará como referencia para el duty cycle de PWM. Para evitar movimientos innecesariamente largos, el error se normalizará al intervalo -180° a +180°.

$$\begin{aligned} \text{setpoint} = 10^\circ, \text{posición} = 350^\circ &\rightarrow \text{error} = +20^\circ \\ \text{setpoint} = 350^\circ, \text{posición} = 10^\circ &\rightarrow \text{error} = -20^\circ \end{aligned}$$

Inicialmente se prevé comenzar con control proporcional o proporcional-derivativo, dejando Ki en cero durante las primeras pruebas, para luego ajustar la acción integral solo si queda error estacionario.

VIII. ARQUITECTURA FREERTOS

El firmware se organizará en tareas. La división propuesta busca aislar las funciones críticas de control y sensado respecto de las tareas de interfaz, comunicación y almacenamiento. El scheduler ejecutará siempre la tarea de mayor prioridad que se encuentre en estado Ready; cuando una tarea espere una cola, semáforo, evento o retardo, quedará bloqueada y no consumirá CPU.

A. Tareas previstas

- PID_Task: recibe comandos, calcula el PID, genera PWM y controla IN1/IN2.
- AS5600_Reader_Task: lee periódicamente el encoder, convierte la lectura a grados y publica la posición.
- I2C_Manager_Task: administra el bus I2C compartido entre sensor y display.
- Safety_Task: supervisa el movimiento y detecta posibles bloqueos.

- UART_Command_Task: recibe e interpreta comandos remotos desde PC.
- ButtonHandler_Task: procesa pulsadores, menú y antirrebote por software.
- Display_Task: actualiza el LCD con menú, estado y mensajes.
- Storage_Task: lee y guarda parámetros en memoria no volátil.

B. Prioridades propuestas

- Storage_Task, prioridad 1: persistencia no crítica.
- Display_Task, prioridad 1: visualización no crítica.
- ButtonHandler_Task, prioridad 1: interfaz local no crítica.
- UART_Command_Task, prioridad 2: comandos remotos.
- Safety_Task, prioridad 2: detección de bloqueo y fallas.
- I2C_Manager_Task, prioridad 3: administración del bus I2C.
- AS5600_Reader_Task, prioridad 3: lectura crítica de posición.
- PID_Task, prioridad 3: control periódico del motor.

IX. COMUNICACIÓN ENTRE TAREAS

La comunicación principal se realizará mediante colas. Las tareas de interfaz local, UART, almacenamiento y seguridad podrán enviar comandos a ControlQueue, que será recibida por PID_Task. La lectura angular se publicará mediante angleQueue, mientras que el estado del motor se enviará a MotorStateQueue para que Safety_Task pueda supervisar el movimiento.

El bus I2C se tratará como un recurso compartido administrado por I2C_Manager_Task. De esta forma se evita que el LCD y el AS5600 intenten acceder simultáneamente al mismo bus. Los pulsadores utilizarán interrupciones GPIO, pero el procesamiento se diferirá a ButtonHandler_Task mediante semáforos.

X. DETECCIÓN DE BLOQUEO

El sistema deberá detectar situaciones en las que el motor recibe PWM pero la posición medida por el AS5600 prácticamente no cambia.

Si se detecta bloqueo, Safety_Task enviará un comando a PID_Task para detener o invertir momentáneamente el sentido de giro, intentando alcanzar el setpoint por el camino alternativo.

XI. CONCLUSIONES

El sistema propuesto permite abordar el control de posición de un motor DC mediante una arquitectura de lazo cerrado basada en ESP32-S3, AS5600 y L298N. La utilización de FreeRTOS resulta adecuada para separar el control, la adquisición de datos, la interfaz de usuario, la comunicación remota, la seguridad y el almacenamiento en tareas independientes.

XII. REFERENCIAS

- [1] R. Barry, Using the FreeRTOS Real Time Kernel: A Practical Guide, documentación de referencia utilizada en los apuntes de Técnicas Digitales III.
- [2] Espressif Systems, ESP-IDF Programming Guide, documentación técnica para desarrollo sobre ESP32-S3.
- [3] Espressif Systems, ESP32-S3 Technical Reference Manual.
- [4] ams OSRAM, AS5600 12-bit Programmable Contactless Potentiometer, datasheet.
- [5] STMicroelectronics, L298 Dual Full-Bridge Driver, datasheet.

ANEXO I - DIAGRAMA DE TAREAS

La Fig. 2 muestra el diagrama de tareas y comunicaciones del firmware. En él se representan las tareas principales, las colas de comunicación, los semáforos utilizados desde interrupciones y la relación entre el control del motor, la lectura del sensor, la interfaz local, la UART y el almacenamiento.

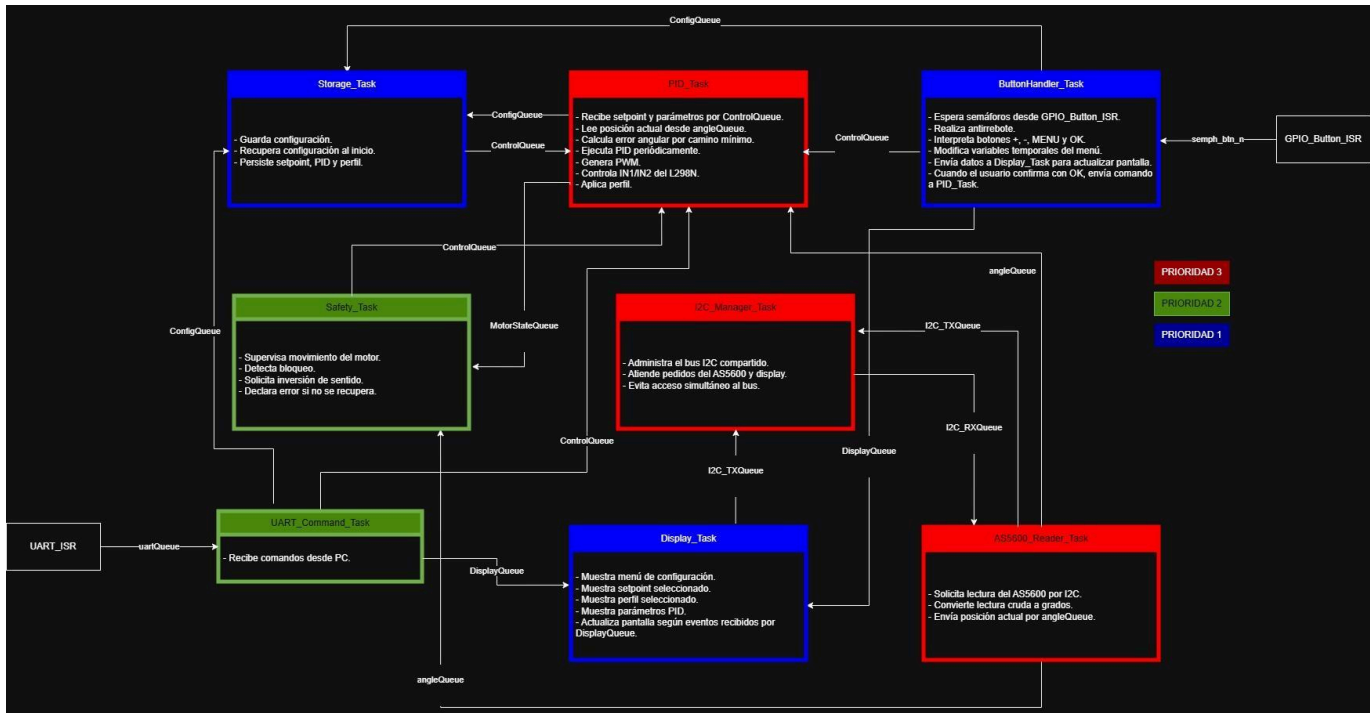


Fig. 2. Diagrama de tareas preliminar del firmware FreeRTOS.